

Session 20

Dart Async Programming & Exception Handling

Objective: Practice Futures, async/await, Future.delayed, exception handling, try-catch, and asynchronous programming in Dart.

Task 1: Fetch Trending Songs

Description: Create a Future that simulates loading trending songs after 2 seconds.

```
Future<String> fetchTrendingSongs() async {  
  await Future.delayed(Duration(seconds: 2));  
  return "Top songs loaded!";  
}  
  
void main() async {  
  print(await fetchTrendingSongs());  
}
```

Expected Output:

Top songs loaded!

Task 2: Order Status

Description: Use async/await with Future.delayed to simulate a Zomato order.

```
Future<String> getOrderStatus() async {  
  await Future.delayed(Duration(seconds: 3));  
  return "Order Delivered";  
}  
  
void main() async {  
  print(await getOrderStatus());  
}
```

Expected Output:

Order Delivered

Task 3: Movie Tickets

Description: Throw an exception if more than 6 tickets are requested.

```
String getMovieTickets(int tickets) {  
    if (tickets > 6) {  
        throw Exception("Cannot book more than 6 tickets");  
    }  
    return "Tickets Booked";  
}  
  
void main() {  
    try {  
        print(getMovieTickets(8));  
    } catch (e) {  
        print("Error: $e");  
    }  
}
```

Expected Output:

Error: Exception: Cannot book more than 6 tickets

Task 4: Payment Process

Description: Randomly succeed or fail using Random().nextBool().

```
import 'dart:math';  
  
void processPayment() {  
    try {  
        if (Random().nextBool()) {  
            throw Exception("Transaction failed");  
        }  
        print("Payment Successful");  
    } catch (e) {  
        print("Payment Failed: $e");  
    }  
}
```

Expected Output:

Payment Successful

or

Payment Failed: Exception: Transaction failed

Task 5: Cricket Scores

Description: Async function generated with AI that fetches scores after a 1.5-second delay.

```
Future<void> fetchCricketScores() async {  
  try {  
    await Future.delayed(  
      Duration(milliseconds: 1500));  
    print("India: 245/6");  
  } catch (e) {  
    print("Error: $e");  
  }  
}  
  
void main() async {  
  await fetchCricketScores();  
}
```

Expected Output:

India: 245/6

Conclusion: This session covered asynchronous programming using Future, async/await, delayed execution, exception handling with try-catch, and simulating real-world API and payment workflows in Dart.